

Rapport définitif de SAE S6 :

Nom du projet : Umshini

Membres du groupe :

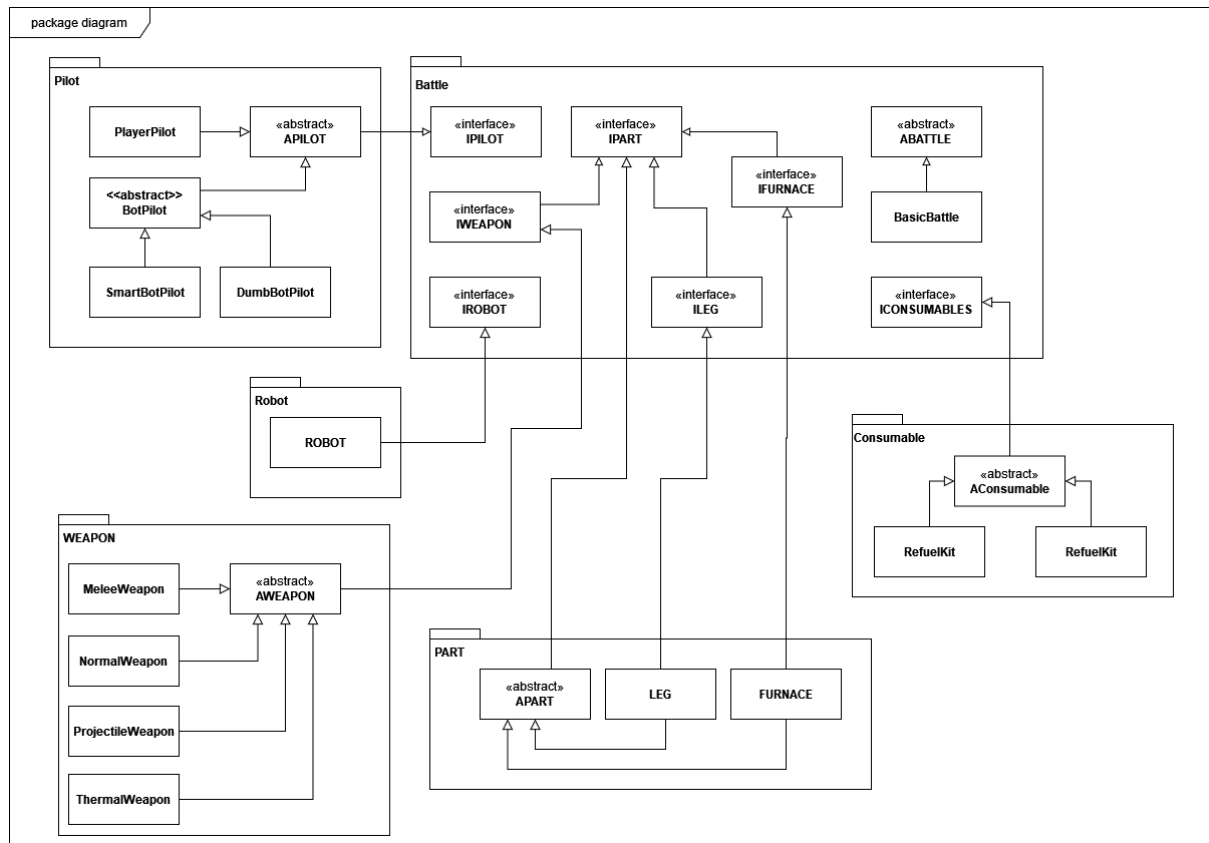
- Martin BIOLLEY,
- Mehdi WAHADA
- Danis TRAVANCIC
- Cyprien MEJAT

Sommaire :

Sommaire :	2
Description de l'existant :	3
Description des maintenances évolutives :	4
Intégration de la GUI :	4
Génération procédurale des robots ennemis :	4
Ajout d'un aspect Roguelike au jeu :	4
Implémentation d'un algorithme de renforcement :	4
Organisation de l'équipe :	5
Démarche de réalisation du projet (rôles, etc.) :	5

Description de l'existant :

Le projet rendu lors de la SAÉ 5 Développement avancé était un jeu tour par tour nommé Umshini prenant la forme d'une application console, l'interface graphique n'ayant pas pu être intégré à temps à cause de problèmes de merge Git et d'une méconnaissance de cet outil de versionning de la part du groupe.



Le diagramme de paquetages de l'architecture du projet représentant uniquement les liaisons externes des paquets

Description des maintenances évolutives :

Intégration de la GUI :

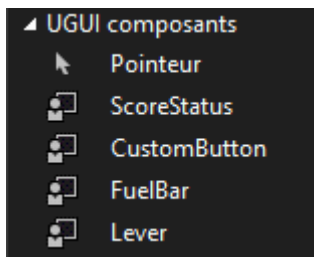


Pour ce projet, nous avons utilisé les Forms de Visual Studio en C#. Cette approche nous a permis de créer une interface graphique riche et interactive pour notre jeu. Dans ces Forms, nous avons intégré plusieurs contrôles pour gérer les différentes parties de l'interface utilisateur.

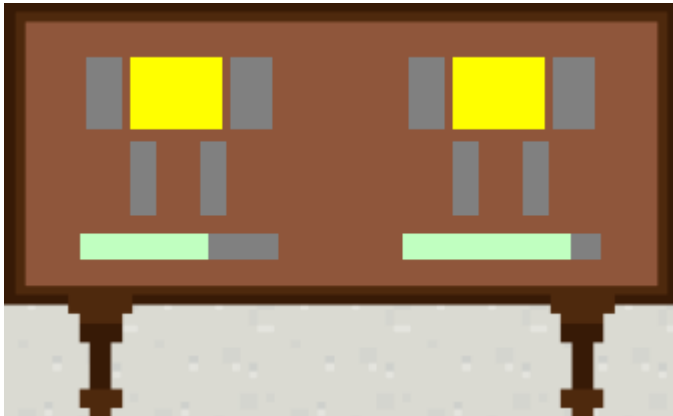
Les contrôles de type "Layout" ont joué un rôle crucial dans l'adaptation responsive de notre interface. Ces contrôles, tels que les `TableLayoutPanel` et `FlowLayoutPanel`, permettent de disposer dynamiquement les éléments en fonction de la taille de la fenêtre. En utilisant les propriétés `Dock` et `Anchor`, nous avons pu garantir que les éléments de l'interface se redimensionnent et se repositionnent correctement lorsque la fenêtre est redimensionnée. La propriété `Dock` permet de fixer un contrôle à un bord de la fenêtre, tandis que la propriété `Anchor` permet de définir les côtés de la fenêtre auxquels un contrôle est ancré, assurant ainsi une adaptabilité maximale.

Les deux robots et le fond d'écran visibles dans l'image de référence sont des images intégrées dans l'interface. Ces éléments visuels contribuent à l'esthétique et à l'immersion du jeu.

Le tableau des scores, les leviers, ainsi que le panneau de notification sont des contrôles personnalisés:



Ces contrôles ont été spécialement conçus et adaptés pour répondre aux besoins spécifiques du gameplay. Par exemple, le tableau des scores affiche la durée de vie des équipements des joueurs ainsi que leur carburant,

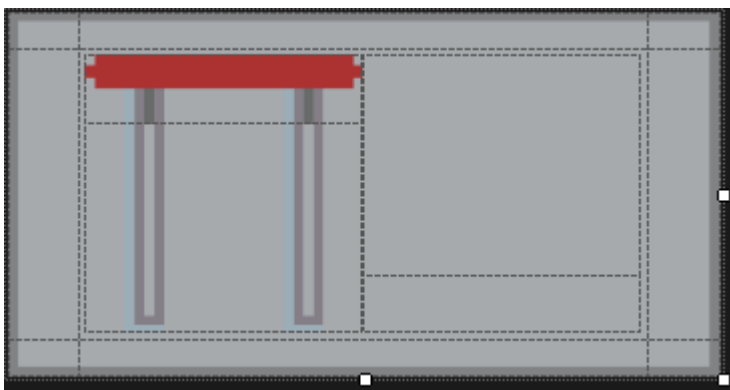


les leviers permettent une interaction intuitive avec le jeu, et le panneau de notification informe les joueurs des événements importants en temps réel.

Cette personnalisation et cette utilisation des Layouts nous ont permis de créer une interface utilisateur adaptée à la façon dont le jeu doit être joué, et en combinant les images et les contrôles personnalisés.

Grâce à la fonctionnalité de création de UserControls dans Visual Studio, nous avons pu développer des contrôles parfaitement adaptés à l'esthétique de notre jeu en manipulant des images. Cela nous a permis de créer des éléments interactifs uniques et visuellement cohérents avec le reste du design du jeu.

Par exemple, pour les levier:



Nous avons utilisé une image représentant le manche rouge qui se déplace de haut en bas. Ce mouvement est limité par des bornes définies pour empêcher le manche

de sortir des limites du levier. L'image fournie de notre UserControl illustre cette personnalisation. On y voit le levier avec le manche rouge et les zones où les limites de mouvement sont définies, ainsi que la zone dédiée au texte, positionnée de manière à s'adapter dynamiquement.

Une particularité de ce contrôle est que l'image du levier rouge "snap" dynamiquement sur le texte positionné à droite. Cela signifie que lorsque le manche est déplacé, il s'aligne automatiquement avec le texte. Cela est rendu possible grâce aux différents Mouse Events disponibles sur Visual Studio C#, qui permettent d'exécuter du code en réponse aux différentes actions de la souris.

```
private void LeverPictureBox_MouseUp(object sender, MouseEventArgs e)
{
    if (_isDragging)
    {
        _isDragging = false;
        LeverPictureBox.Capture = false;

        // Snap to the closest label
        Label closestLabel = null;
        int minDistance = int.MaxValue;

        foreach (Label label in _lLabelList)
        {
            int distance = Math.Abs(LeverPictureBox.Top - label.Top);
            if (distance < minDistance)
            {
                minDistance = distance;
                closestLabel = label;
            }
        }
    }
}
```

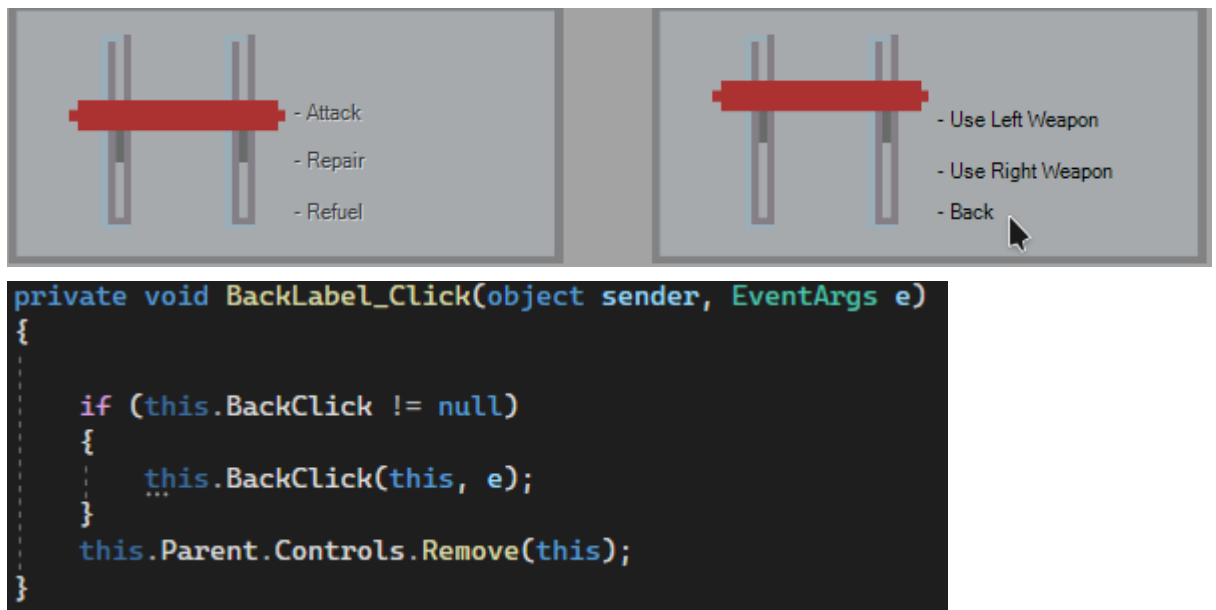
Les leviers et leurs textes sont générés dynamiquement les uns après les autres avec leur propres "events" grâce à la fonction "CreateNewLever()":

```
private void CreateNewLever(List<string> lLabelsText)
{
    Lever newLever = new Lever(lLabelsText);

    if (this.generalLayout != null)
    {
        TableLayoutPanelCellPosition posLayoutLever = generalLayout.GetCellPosition(_lLever.Last());
        this.generalLayout.Controls.Add(newLever, posLayoutLever.Column + 1, posLayoutLever.Row);
    }

    newLever.LabelClick += new EventHandler(lever_LabelClick);
    newLever.BackClick += new EventHandler(lever_BackClick);
    newLever.Dock = DockStyle.Fill;
    newLever.Margin = leverMainMenu.Margin;
    _lLever.Last().Enabled = false;
    _lLever.Add(newLever);
}
```

et lorsque le levier est aligné sur "-Back (Revenir en arrière)", il est automatiquement enlevé de l'interface:



Les différentes animations de l'interface sont réalisées par des déplacements d'images. Par exemple, un tremblement peut être utilisé pour simuler un choc du robot

Menu:



Le menu principal, tout comme l'interface de combat, est conçu à l'aide de layouts et de contrôles personnalisés, notamment des boutons.

Génération procédurale des robots ennemis :

L'idée générale de la génération procédurale est de créer du contenu numérique de manière automatique selon des procédés bien définis.

Dans notre cas, la génération procédurale sera faite de manière aléatoire.

Pour cela, des objets C# représentant les composants d'un robot ont été sérialisés via les méthodes de la librairie Newtonsoft et stockés dans un fichier JSON dans lequel chaque type de composants est stocké dans un tableau.

Les données du fichier seront utilisées dans deux méthodes essentielles pour la génération du robot :

- GetProceduralRobot(string) récupère les données du fichier JSON, récupère la taille de chaque liste et génère un entier défini aléatoirement qui permettra de sélectionner les composants du robot ;
- GetWeapon() renvoie une arme d'un type défini en fonction du type renseigné dans l'objet JSON ;

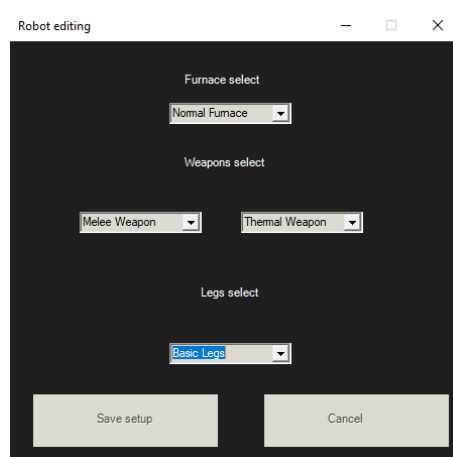
Ainsi, quand une partie de jeu est lancée, le robot est généré via ces deux méthodes.

Personnalisation de l'équipement du joueur :

Si le robot est généré de manière procédurale, on laisse le choix au joueur de participer avec un équipement prédéfini ou de choisir les composants que son robot aura.

Pour ce faire, un formulaire (montré dans l'image ci-dessous) a été réalisé afin que le joueur puisse équiper son robot de la manière dont il le souhaite. Chaque champ est respectivement renseigné avec le nom des fournaises, armes et jambes du robot que l'on souhaite personnaliser.

Une fois sauvegardé via le bouton "Save setup", les informations du robot personnalisé seront transférées au formulaire de départ qui permettra de jouer notre robot personnalisé.

The image shows a software window titled "Robot editing" with standard Windows window controls (minimize, maximize, close). The window contains a dark-themed form with three sections: "Furnace select" with a dropdown menu showing "Normal Furnace"; "Weapons select" with two dropdown menus, "Melee Weapon" and "Thermal Weapon"; and "Legs select" with a dropdown menu showing "Basic Legs". At the bottom of the form are two buttons: "Save setup" and "Cancel".

Le formulaire de personnalisation du robot

Implémentation d'un algorithme d'apprentissage par renforcement :

Le développement de *Smart_Bot* dans la partie 1 du projet avait pour objectif de créer un "anti-joueur" qui pouvait battre le joueur avec la stratégie supposément la plus optimisée possible.

Dans cette partie 2, le choix a été fait de développer un robot (un agent) qui cherche à battre ce *Smart_Bot*, en les faisant affronter des centaines de milliers de fois, afin que ce "IA Bot" puisse trouver une éventuelle faille dans la stratégie de *Smart_Bot*, ou une faille d'exploitation dans le gameplay lui-même.

Le choix a été fait d'implémenter le Q-Learning, car le plus utilisé dans la recherche pour créer des IA battant des humains, mais aussi par la grande quantité de ressources sur Internet (notre parcours ne nous a donné aucune formation sur tout ce qui concernait l'apprentissage par renforcement, il s'agissait donc là d'un critère de choix très discriminant).

L'apprentissage par renforcement du Q-Learning repose sur le principe de matrice de récompense et de Q Table qui va donner à l'agent un arbre probabiliste pondéré en fonction de ses actions par rapport à son état.

3 types différents de calcul d'état ont alors été effectués, s'améliorant au fil du temps :

- La première version, *State2*, repose sur la transposition des paramètres en binaire.

Chaque robot a en effet quatre parties du corps (leur four, leurs jambes, leurs deux armes), et un niveau de fuel, les cinq représentés par deux booléens afin d'avoir plus d'informations pour le robot. Cela donne 2^{10n} possibilités d'état, avec n le nombre de robots.

Avec deux robots, cela donne près de 1 000 000 d'états possibles. Autant dire qu'une optimisation était nécessaire...

- La deuxième version, *State3*, repose sur la transposition des paramètres en trinaire.

Il repose sur les mêmes informations que *State2*, mais transforme les 2 booléens de la même partie du robot (*armure_detruite* et *partie_detruite*) en une seule partie à 3 états possibles (l'état *armure_intacte* et *partie_detruite* étant impossible, il n'y a pas de perte d'information). Cela donne 3^{5n} possibilités d'état, avec n le nombre de robots. Avec deux robots, cela donne près de 60 000 d'états possibles. Une économie d'un facteur 18 a donc été réalisé par cette simple révision.

- Une troisième version, *State_Death_Bot*, voulait insister sur le fait qu'un robot détruit/inutilisable soit représenté par un seul état (ce qui n'était pas le cas avant).

La formule révisée donne donc un nombre d'états égal à $3^{4n} \times 2^n + (n+1)$ états possibles. Avec deux robots, cela donne près de 26 000 d'états possibles.

Cette formule est donc 2.3 fois plus en économe d'états par rapport à *State3*, et 38 fois par rapport à *State2*.

Seul *State_Death_Bot* est encore pertinent pour les tests à ce jour, *State2* et *State3* servant justant à montrer tout le chemin accompli.

Il est d'ailleurs intéressant de remarquer qu'il y a des gains de performance (à la marge, de l'ordre de 0,3%, mais significatifs au vu du nombre d'itérations), sur les tests menés entre *State_Death_Bot* et *State3*.

De plus, même si la matrice finale générée par l'algorithme est au final plus grande, elle obtient en échange une précieuse information : peu importe l'état des autres pièces, si le four (le cœur du robot) est détruit, c'est la fin de la partie (et vous perdez un point). Nous supposons en effet que cela pouvait aider l'algorithme à gagner un nombre précieux d'entraînements à effectuer.

Afin de ne pas devoir modifier la classe *QLearning* à chaque modification de calcul de la matrice d'état, une interface *IState* a été développé, couplé à une *Factory*. La classe *QLearning* passe donc vers une implémentation en composition, comme vu en 2^e année.

Au sujet de la matrice de récompense, l'objectif était de donner des récompenses uniquement lorsque l'agent "IA Bot" réussissait à tuer le robot ennemi. Si jamais l'agent était détruit, ou s'il était incapable de faire la moindre action, par exemple par un manque de carburant ou de munition, il perdait un point.

Il était évidemment possible de multiplier les récompenses suivant les cas (en cas de rechargement de son fuel, par exemple), mais l'objectif restait qu'il trouve un moyen de tuer son adversaire, et rien d'autre, en lui laissant le plus d'initiative possible.

En raison du grand nombre d'états, la matrice de récompense s'est d'ailleurs transformée en une méthode qui analyse le cours de la partie pour retourner une éventuelle récompense, ce qui au final revient au même, et est moins couteux en espace mémoire.

Le *QLearning* se distingue également par sa catégorie "Action", représenté ici sous forme d'énumération. En raison de son manque d'élasticité (on ne compte pas supprimer ou ajouter des actions à l'avenir), aucun découplage par rapport à la classe *QLearning* n'a été fait.

En revanche, une interface *IQLearning* a été créé dans le cas où un besoin radical de fonctionnement de l'algorithme se ferait ressentir.

Durant les tests de simulation, il est apparu que le manque de fuel décimait les duels, la consommation de carburant a donc largement été revue à la baisse.

Il est apparu par ailleurs (après l'oral de la partie 2 de la SAE) que les chances de victoires de l'agent étaient considérablement augmentées lorsque les points de vie des fours étaient

augmentés : on passe alors de 4 victoires sur 100 000 parties à plus de 74 000 sur 100 000 (taux de victoire oscillant entre 74 et 75%).

L'analyse des actions du robot n'ont pas permis de comprendre sa technique, qui semble proche de l'aléatoire.

Ce test va en l'encontre de la conclusion prononcé lors de l'oral de la SAE : il semble que le nombre d'états était de base suffisant, mais que le temps des combats (en raison de la vie des fours) limitait la capacité d'apprentissage de l'agent.

```
Training Agent...
Training is Done!

Ready for run, please wait...
Victories : 74466
Defeats : 25534
Agent win rate : 74,47%
```

Capture d'écran montrant le résultat de l'apprentissage par renforcement

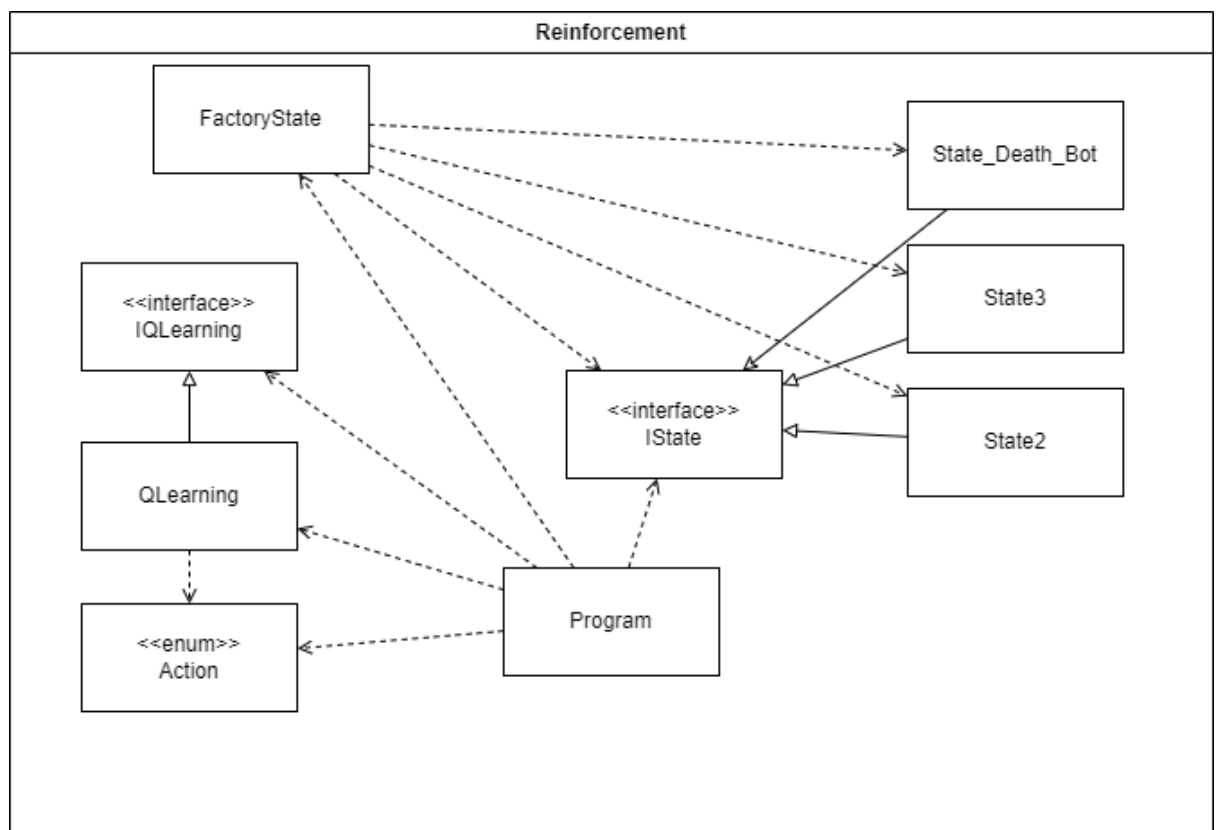


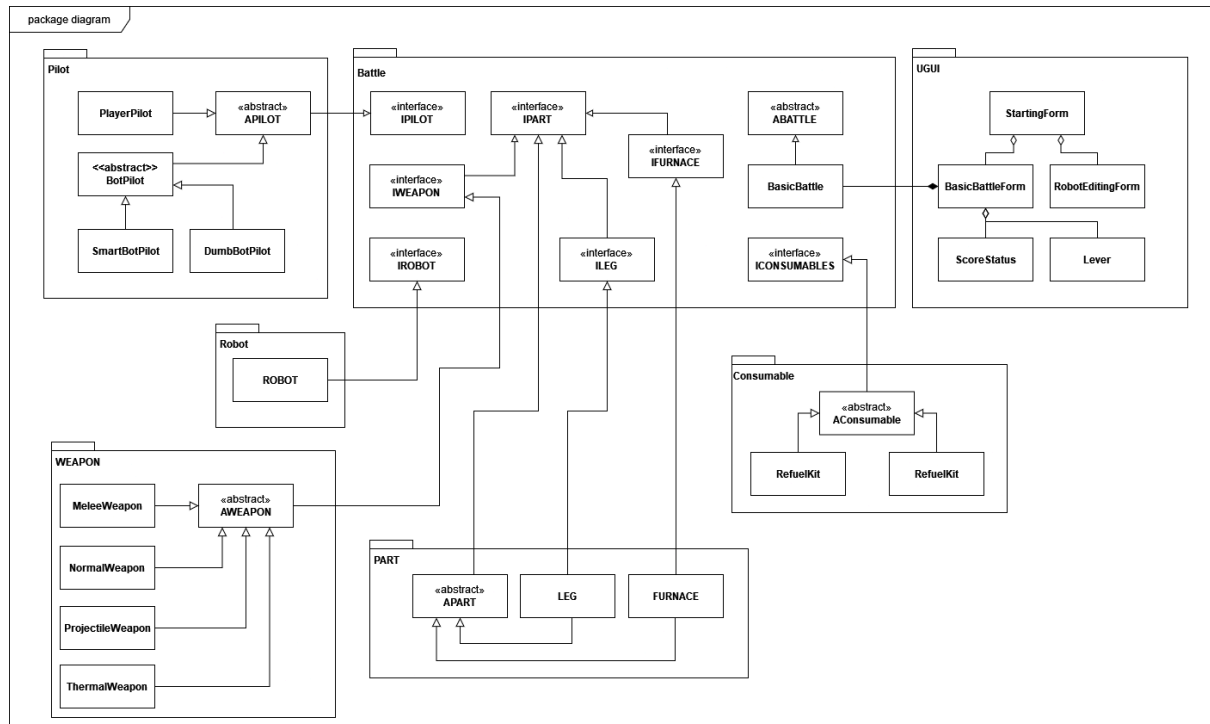
Diagramme UML de dépendance

Organisation de l'équipe :

Démarche de réalisation du projet (rôles, etc.) :

- Martin : Code reviewer et Développeur support. Durant la SAÉ S5, Martin était Architecte et se chargeait des reviews de codes ainsi que de la création des ressources graphiques et sonores nécessaire à la création du jeu, il s'est aussi chargé de réaliser le premier POC du jeu, pour avoir une référence.
- Mehdi se chargeait du développement de la génération procédurale du robot ainsi que du développement du formulaire de personnalisation du robot. Durant la SAÉ S5, il s'était occupé des tests unitaires de chaque fonctionnalité importante au bon fonctionnement du jeu ;
- Cyprien : S'est chargé de se documenter sur les algorithmes d'apprentissage par renforcement, puis sur le Q-Learning, et a implémenté cet algorithme. S'est ensuite concentré sur l'optimisation de la taille de la matrice d'état avec succès, la passant de 1 000 000 d'états à 26 000.
- Danis : S'est chargé de réaliser les interfaces graphiques, les animations et l'adaptation responsive du jeu. Il a également été responsable d'intégrer les effets sonores et de la création de contrôles responsives, pour que le jeu soit intuitif et esthétiquement plaisant, tout en étant entièrement fonctionnel. Durant la SAE S5, il s'est chargé de l'adaptation de la POC initialement développée en C++ vers le langage C#.

Description technique :



Le diagramme de packages de l'architecture du projet représentant uniquement les liaisons externes des paquets